



## Chapter 2: The Windows App SDK for a Windows Forms Developer

In the previous chapter, we learned the basics of the Windows App SDK. Now, it's time to put the technology into practice by learning how to use it to modernize our existing applications.

Windows Forms is, without any doubt, one of the most popular development technologies in the desktop ecosystem. It was the first desktop platform that was introduced with .NET Framework, and it has remained popular over the years, especially in the enterprise space, thanks to its easy learning curve. Owing to a powerful designer, it's relatively easy to design complex user interfaces, handle user interactions, and perform complex tasks.

Over the course of the years, however, it also started to show multiple limitations:

- Windows Forms was released almost 20 years ago, and as such, it isn't been able to handle modern scenarios, such as high-resolution screens and multiple input types.
- The importance of user experience and good design has grown significantly in the last decade. Windows Forms was designed in a different era, and it doesn't provide many ways to deeply customize the look and feel of an application.
- The **user interface (UI)** is deeply connected with the business logic, making it hard to apply patterns that can help to increase the testability, maintainability, and supportability of a project.

These are some of the challenges that led Microsoft to rethink the desktop development experience by introducing a different UI paradigm with **Windows Presentation Foundation (WPF)** in 2006. With the advent of XAML, Microsoft created a UI platform based on a markup language, which enables developers to tackle many of the challenges they faced with Windows Forms – better customization options, theming support, separation of concerns, and so on.

These are the reasons why XAML is still at the heart of many Microsoft development technologies, including WinUI. As such, this chapter is dedicated to all the Windows Forms developers or, generally speaking, developers who are coming from different platforms. We'll explore all the most important XAML concepts, from the basics to the most advanced ones, which are critical to building beautiful and robust applications and taking advantage of all the powerful features offered by WinUI.

In this chapter, we'll explore the following concepts:

- What XAML is and how it works
- XAML namespaces
- Properties and events
- Resources, styles, and templates
- Animations
- Data binding
- Data templates
- Creating user controls

In ***Chapter 7**, Migrating Your Windows Applications to the Windows App SDK and WinUI*, we'll learn how to put in practice many of these concepts to migrate a real Windows Forms application.

## Technical requirements

As a reference, we'll use a sample Windows Forms application that you will find on GitHub at the following URL:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter02>

## Introducing XAML

**XAML** (which is the acronym of **Extensible Application Markup Language**) is a markup language based on XML to define the user interface of an application. The main difference compared to Windows Forms is that XAML enables a declarative approach to define a user interface, which makes it easier for a developer to interpret it just by looking at the code.

For example, consider the following UI:

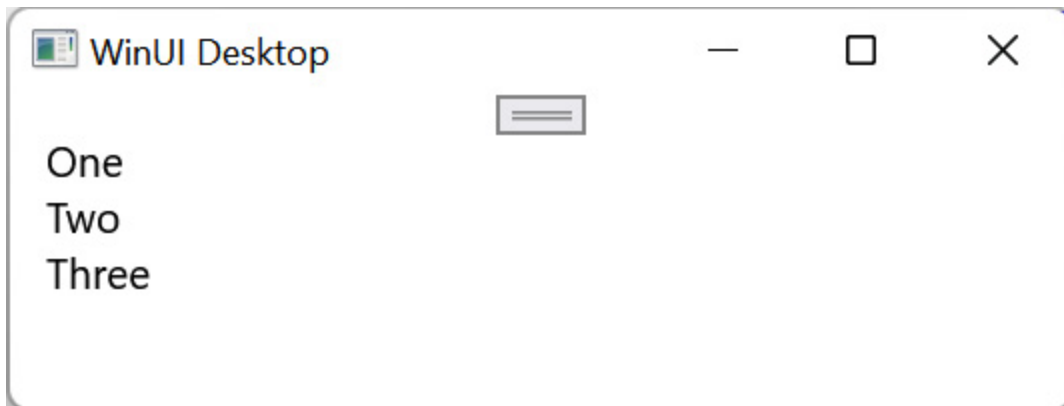


Figure 2.1 – A simple WinUI application

This is how you describe it using XAML:

```
<Window>
  <StackPanel>
    <TextBlock>One</TextBlock>
    <TextBlock>Two</TextBlock>
    <TextBlock>Three</TextBlock>
```

```
</StackPanel>
</Window>
```

Alternatively, this is how the same UI can be described using C#, as Windows Forms does under the hood when you're working with the designer:

```
public partial class App : Application
{
    private Window m_window;
    private StackPanel;
    private TextBlock textBlock1;
    private TextBlock textBlock2;
    private TextBlock textBlock3;
    protected override void
OnLaunched(Microsoft.UI.Xaml.
    LaunchActivatedEventArgs args)
    {
        m_window = new Window();
        stackPanel = new StackPanel();
        textBlock1 = new TextBlock();
        textBlock2 = new TextBlock();
        textBlock3 = new TextBlock();
        textBlock1.Text = "One";
        textBlock2.Text = "Two";
        textBlock3.Text = "Three";
        stackPanel.Children.Add(textBlock1);
        stackPanel.Children.Add(textBlock2);
        stackPanel.Children.Add(textBlock3);
        m_window.Content = stackPanel;
        m_window.Activate();
    }
}
```

You will immediately note how the XAML version isn't just more concise; it's also easier to read and interpret. Even if you don't have experience with XAML-based technologies, the hierarchy defined by the

XML structure makes it easier to understand the relationship between all the various items in the UI – we have a window, which hosts a panel, which includes three text controls. Understanding the same structure by reading the C# code instead is a tougher challenge.

## Using namespaces

If you look at the top-level definition of an XAML file, such as **Window** or **Page**, you will find multiple namespaces declared as attributes, as shown in the following examples:

```
<Window
  x:Class="MyApp.MainWindow"
  xmlns="http://schemas.microsoft.com/
    winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:App14"
  xmlns:d="http://schemas.microsoft.com/
    expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-
    compatibility/2006"
  mc:Ignorable="d">
</Window>
```

The role of namespaces in XAML is the same as in C# – they provide a path that tells the compiler where a specific class is available. All the standard controls and properties provided by WinUI are included in the default platform namespace. This means that you can use controls such as **Button**, **TextBlock**, or **Grid** without having to specify any prefix, as shown here:

```
<TextBlock>One</TextBlock>
```

Instead, if you need to use in your XAML file a class or a control that isn't a part of the framework (for example, it's coming from a NuGet library or it's a custom control you have built), you first have to declare

its namespace. For example, let's say that you want to use the following control (we'll learn how to build one later in the *Creating user controls* section):

```
namespace MyApp.Controls
{
    public sealed partial class MyUserControl :
        UserControl
    {
        public MyUserControl()
        {
            this.InitializeComponent();
        }
    }
}
```

As you can see, this class is declared as part of the **MyApp.Controls** namespace. As such, before using it, we have first to declare this namespace in our XAML file and assign it an alias by using the **xmlns:controls** syntax, as in the following snippet:

```
<Window
    x:Class="MyApp.MainWindow"
    xmlns:controls="using:MyApp.Controls">
</Window>
```

Then, to use the control inside your XAML file, you must add the alias as a prefix, as shown in the next example:

```
<Window
    x:Class="MyApp.MainWindow"
    xmlns:controls="using:MyApp.Controls">
    <controls:MyUserControl />
</Window>
```

Let's see now how we can interact with these controls.

## Customizing controls with properties

Every control offers multiple properties that you can use to customize their look, feel, and behavior. For example, a **TextBlock** control offers properties to define the font size, the text color, or the font-weight. Properties are declared as XML attributes, as in the following example:

```
<TextBlock Text="Hello world" FontSize="18" />
```

Properties can be declared in an explicit way as children of the control, as shown in the following snippet:

```
<TextBlock Text="Hello world">
    <TextBlock.FontSize>18</TextBlock.FontSize>
</TextBlock>
```

For simple properties such as **FontSize**, there isn't any added value in using the extended syntax. However, there are more complex properties that can be only set explicitly. For example, let's say that you want to set the background color of a **Button** control with a gradient. In this case, a simple property isn't able to satisfy this requirement, so you'll have to configure it using the following code:

```
<Button Content="Ok">
    <Button.Background>
        <LinearGradientBrush StartPoint="0,0"
            EndPoint="1,0">
            <LinearGradientBrush.GradientStops>
                <!-- Collection type -->
                <GradientStop Offset="0.0" Color="Blue"
            />
                <GradientStop Offset="0.5"
            Color="White" />
                <GradientStop Offset="1.0" Color="Red"
            />
            </LinearGradientBrush.GradientStops>
        </LinearGradientBrush>
    </Button.Background>
```

```
</Button>
```

An important property exposed by every control is **x:Name**, which is used to create a reference to the control that you can leverage from the code-behind class. For example, let's add this property to a **TextBlock** control, as shown here:

```
<TextBlock x:Name="txtMessage" />
```

Now, you'll be able to change any property of the control or access any event or method directly in the code-behind class, using the following syntax:

```
txtMessage.Text = "Hello world!";
```

This approach should be familiar if you're a Windows Forms developer, since it works in a very similar way – you assign a name to the control via the designer, and then, in code, you can reference it and customize its properties.

## Reacting to changes with event handlers

Most of the XAML controls offer events, which are triggered when a specific condition happens. A very simple example can be found in the **Button** control, which exposes the **Click** event that is triggered when a user clicks on the button. Events also support more complex scenarios – a **TextBox** control exposes an event to notify when text has changed, and a **Window** exposes events to notify when its position or size has changed.

All these scenarios can be managed with an event handler, which is a special method that gets invoked when the event is triggered. Thanks to a handler, you can react to the event in the most appropriate way.

The syntax to define an event handler is the same as the one used for properties. The only difference is that the value of the property must match the name of a method declared in the code-behind class. The IntelliSense feature provided by Visual Studio can generate one for



you if it doesn't exist – for example, this is the code you can use to manage the **Click** event exposed by **Button**:

```
<Button Click="OnButtonClicked" />
```

This will generate the following event handler in the code-behind class:

```
private void OnButtonClicked(object sender,  
    RoutedEventArgs e)  
{  
    //do something  
}
```

As you can see, an event handler isn't a simple method, but it carries two fixed parameters:

- The **sender**, which is a reference to the control that raised the event. In the previous example, the **sender** object will contain a reference to the **Button** control.
- The event arguments, which contain additional information about the event context – for example, if you're working with a list, the event arguments will contain a reference to the item selected by the user.

This special signature is the reason why event handlers are deeply connected with the UI – they can be declared only in the code-behind class. You can't declare an event handler in an external class. As we're going to learn in [\*Chapter 6, Building a Future-Proof Architecture\*](#), we can use different techniques to break this tight connection so that we can make our applications easier to test and maintain.

Also, in this case, event handlers should already be familiar to Windows Forms developers, except for the declaration syntax. Event handlers can also be declared in code – in this case, the syntax is the same as Windows Forms:

```
public MainWindow()  
{  
    this.InitializeComponent();  
    btnMyButton.Click += OnButtonClicked;  
}  
private void OnButtonClicked(object sender,  
    RoutedEventArgs e)  
{  
    //do something  
}
```

Now that we have learned the basic XAML concepts, we can start enhancing our controls with resources.

## Styling your applications with resources

Resources are one of the most powerful features of XAML, since they simplify the adoption of a unified theme across a whole application. Thanks to resources, you can define a snippet of XAML and reuse it across multiple controls in an application.

For example, let's say that you want a series of **TextBlock** controls to display text in red. Instead of setting the **Foreground** property of each control to **Red**, you can define a resource with this setting, as shown in the following example:

```
<StackPanel>  
    <StackPanel.Resources>  
        <SolidColorBrush x:Key="MyBrush" Color="Red" />  
    </StackPanel.Resources>  
</StackPanel>
```

Resources are defined by a special property called **x:Key**. It's the name we're going to use to reference this resource on our XAML page.

To apply a resource to a property, we need to introduce the concept of **markup expressions**. These are special expressions that you can use in your XAML code that, under the hood, use potentially complex logic to resolve a value. Markup expressions are set like a normal property, but they are included inside brackets. To apply a resource, you must use a markup expression called **StaticResource**, which will navigate through the XAML tree to find the right resource to apply, as shown in the following example:

```
<StackPanel>
    <StackPanel.Resources>
        <SolidColorBrush x:Key="MyBrush" Color="Red" />
    </Grid.Resources>
    <TextBlock Text="Hello world!"
        Foreground="{StaticResource MyBrush}" />
</StackPanel>
```

The **StaticResource** markup expression must be followed by a resource key; in the previous example, it was **MyBrush**.

The advantage of using a resource compared to setting the **Foreground** property directly is the reusability it gives. Let's say that the same **Grid** contains three different **TextBlock** controls. If, at any point during the life cycle of your project, you need to change the color from red to green, you just need to change the resource once instead of changing the property on every single control.

In the previous snippet, you might have noticed how the resource was defined inside a property called **Resources**, exposed by the **Grid** control. This property is exposed by every XAML control, so resources can be defined at any level. One of the most powerful features of the **StaticResource** markup expression is that it uses a hierarchical navigation approach to search for a resource. In the previous example, since the resource is declared at **Grid** level, every child control will be able to use it. For this reason, resources are typically defined at the

highest possible level (for example, **Page**) so that every nested control can use them.

This approach also makes it possible to override specific resources if you need deeper customization. The **StaticResource** markup expression will resolve the resource using the closest one it finds in the hierarchy. For example, let's consider the following XAML code:

```
<Page>
  <Page.Resources>
    <SolidColorBrush x:Key="MyBrush" Color="Red" />
  </Page.Resources>
  <StackPanel>
    <StackPanel.Resources>
      <SolidColorBrush x:Key="MyBrush"
Color="Green"
      />
    </StackPanel.Resources>
    <TextBlock Text="Hello world!"
      Foreground="{StaticResource MyBrush}" />
  </Grid>
</Page>
```

As you can see, we are defining two resources with the same name, which is **MyBrush**. Can you guess which will be the final output of the XAML code?

The answer is green because the **StaticResource** markup expression will use the closest resource to the **TextBlock** control, which, in this case, is the one defined at **Grid** level.

## Sharing a resource with a whole application

In the previous section, I mentioned that resources are typically declared at the highest possible level, such as **Page**, so that all the nested

controls can use them. But what if I want to share a resource across an entire application?

The **Application** class (defined in the **App.xaml** file), which acts as a global container, offers a **Resources** property as well, where you can declare resources that can be accessed by every **Window** or **Page** of the application, as shown in the following example:

```
<Application>
  <Application.Resources>
    <SolidColorBrush x:Key="MyBrush" Color="Red" />
  </Application.Resources>
</Application>
```

## Using dictionaries to organize resources

Declaring resources directly in the **Application** class or inside **Page** can generate confusion in the long term, especially if a project grows and becomes more complex. As such, XAML supports the concept of a **resource dictionary**, which are special XAML files that can contain one or more resources. They are different from a traditional XAML page because they don't have a code-behind class, since they aren't meant to contain any logic.

Thanks to resource dictionaries, you can easily include all your resources in a dedicated file, or even create multiple files to organize your resources in a more logical way. To create a resource dictionary, right-click on your WinUI project and choose **Add | New item**. You will find a template called **Resource Dictionary (WinUI 3)** – give a name to the file and click **Ok**.

This is what a resource dictionary looks like:

```
<ResourceDictionary
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml
  /presentation"
```

```
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:MyApp">
<SolidColorBrush x:Key="MyBrush" Color="Red" />
</ResourceDictionary>
```

The file is then registered as an application resource in the **App.xaml** file:

```
<Application>
  <Application.Resources>
    <ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <XamlControlsResources xmlns=
          "using:Microsoft.UI.Xaml.Controls" />
        <ResourceDictionary Source=
          "MyResources.xaml" />
      </ResourceDictionary.MergedDictionaries>
      <!--Other app resources here -->
    </ResourceDictionary>
  </Application.Resources>
</Application>
```

The **Resources** property offers a special property called **ResourceDictionary**, which you can use to declare all the global resources you want across the application. All the resource dictionary files must be declared inside the **MergedDictionaries** property, which, at runtime, will merge all the dictionaries inside a single file. In the previous example, we have registered a single **ResourceDictionary**, but we could have registered as many as we need.

Now that we have learned the basics of how to use resources, let's see how we can use them for more complex scenarios.

## Reusing multiple resources with styles

So far, we have explored simple resources, which are made by a single XAML element. However, XAML supports also more complex types of resources, such as styles. If you are familiar with web development, XAML styles and CSS styles are similar, since they serve the same purpose – group together a set of resources so that you can apply all of them just by referencing the style.

This is an example of what a style looks like:

```
<Style x:Key="HeaderText" TargetType="TextBlock">
    <Setter Property="Foreground" Value="Red" />
    <Setter Property="FontSize" Value="24" />
</Style>
```

As with a single resource, a style is identified by a name assigned to the **x:Key** property. However, we can also see a couple of important differences:

- We must specify a property called **TargetType**, which specifies the control type that the style is targeting.
- Since a style is used to change multiple properties at once, we have one **Setter** entry for each property we want to customize. Each **Setter** specifies the name of the property (through the **Property** attribute) and the value we want to assign (through the **Value** attribute).

The previous example defines a style called **HeaderText** that can be applied to **TextBlock** controls, which will make the text red and with a size of 24. To apply this style, we can reuse the **StaticResource** markup expression. This time, however, we won't assign the resource to a specific property but the **Style** property exposed by all the XAML controls, as shown in the following example:

```
<TextBlock Text="Hello world!" Style="{StaticResource
    HeaderText}" />
```

Styles can also be set in an implicit way by omitting the **x:Key** property as shown:

```
<Style TargetType="TextBlock">
    <Setter Property="Foreground" Value="Red" />
    <Setter Property="FontSize" Value="24" />
    <Setter Property="FontWeight" Value="Bold" />
</Style>
```

In this case, you won't have to manually set the **Style** property on each control, but all the controls with a type that matches the one specified in the **TargetType** property will get a style automatically applied. In the previous example, the style will be applied to all the **TextBlock** controls.

Styles also support inheritance, thanks to the **BasedOn** property. Consider the following example:

```
<Page.Resources>
    <Style x:Key="HeaderText" TargetType="TextBlock">
        <Setter Property="Foreground" Value="Red" />
        <Setter Property="FontSize" Value="24" />
    </Style>
    <Style x:Key="MainTitle" TargetType="TextBlock"
        BasedOn="{StaticResource HeaderText}">
        <Setter Property="FontWeight" Value="Bold" />
    </Style>
</Page.Resources>
```

We have defined two styles, **HeaderText** and **MainTitle**. The second one, however, extends the first one, by setting an extra property, **FontWeight**. As such, the result is that, when we apply the **MainTitle** resource, we'll display the text in red with a font size of 24 (settings are inherited from the **HeaderText** style) and in bold (which is set by the style itself).



# Understanding animations

Animations are another feature that makes XAML quite powerful. They are a special type of resource that enables developers to animate any property of any control in the XAML page by creating a transition that will gradually change its value over time. What makes animations very powerful is that you don't have to manually handle timers, custom drawing, and so on. You just specify which kind of property you want to animate, which are the starting and end values, and for how long the animation must run. The XAML framework will take care of everything else for you by generating all the frames that are needed to create the animation.

Let's see an example:

```
<Page>
  <Page.Resources>
    <Storyboard x:Name="EllipseAnimation">
      <DoubleAnimation
        From="1"
        To="0"
        Duration="0:0:3"
        Storyboard.TargetName="MyEllipse"
        Storyboard.TargetProperty="Opacity" />
    </Storyboard>
  </Page.Resources>
  <StackPanel>
    <Ellipse Height="80" Width="80" Fill="Red"
      x:Name="MyEllipse" />
  </StackPanel>
</Page>
```

Animations are declared as resources using the **Storyboard** object, which can contain one or more animations that are triggered at the same time. There are multiple types of animations in XAML based on the type of property you want to animate. In the previous example, we're acting on the **Opacity** property, which is a number, so we use a **DoubleAnimation** object. Other examples of animations are **ColorAnimation**, **PointAnimation**, and **ObjectAnimation**.

Regardless of the type you choose, an animation is defined by the following properties:

- **From:** The value of the property when the animation starts. It's optional – if you don't specify it, the animation will use the current value as the starting point.
- **To:** The value of the property when the animation ends. This one is optional as well – in this case, the animation will use the current value as the final point.
- **Duration:** The length of the animation. The runtime will automatically generate the animation's frames based on the duration.
- **Storyboard.TargetName:** The name of the control that you want to apply the animation to.
- **Storyboard.TargetProperty:** The name of the property that you want to animate.

The previous storyboard generates a fade-out animation – the **Opacity** property of the control will change from **1** (fully visible) to **0** (hidden) in 3 seconds.

All the animations also offer a variant implementation with the **UsingKeyFrames** suffix, which puts you in control of the animation timeline. In the previous scenario, the frames of the animation were uniformly split across the duration, creating a smooth effect. In some cases, you might instead want to distribute the frames in a different way – for example, you may want the animation to be faster at the beginning and slow down toward the end.

The following snippet shows how you can achieve this goal:

```
<Page>
  <Page.Resources>
    <Storyboard x:Name="EllipseAnimation">
      <DoubleAnimationUsingKeyFrames
Duration="0:0:5"
Storyboard.TargetName="MyEllipse
"
Storyboard.TargetProperty="Opacit
y">
        <LinearDoubleKeyFrame Value="1" KeyTime="0:0:0"
/>
        <LinearDoubleKeyFrame Value="0.8"
KeyTime="0:0:1" />
        <LinearDoubleKeyFrame Value="0.3"
KeyTime="0:0:2" />
        <LinearDoubleKeyFrame Value="0.1"
KeyTime="0:0:4" />
        <LinearDoubleKeyFrame Value="0" KeyTime="0:0:5"
/>
      </DoubleAnimationUsingKeyFrames>
    </Storyboard>
  </Page.Resources>
  <StackPanel>
    <Ellipse Height="80" Width="80" Fill="Red"
x:Name="MyEllipse" />
  </StackPanel>
</Page>
```

We're using the **DoubleAnimationKeyFrame** object, which, in this case, doesn't need to specify the **From** and/or **To** properties, since we'll control the timeline manually. Inside the object, we add multiple **LinearDoubleKeyFrame** objects, which represent a different step in the timeline. Each of them has two properties:

- **Value:** This is the value that the property will assume in that specific step.
- **KeyTime:** This is the exact point in the timeline when the property will assume the value we have specified.

The previous example still generates a fade-out effect but this time with a different transition – the ellipse will start to fade out fast and then it will slow down at the end.

## Playing an animation

You might have noticed how storyboards are a special type of resource. They are declared inside the **Resources** collection of a control or page, but they are identified by the **x:Name** property rather than the **x:Key** one.

This means that, in the code-behind, they can be referenced like a regular control. As such, all you have to do to start an animation is to invoke the **Begin()** method, as shown in the following example:

```
private void OnStartAnimation(object sender,
    RoutedEventArgs e)
{
    EllipseAnimation.Begin();
}
```

You can subscribe to the **Completed** event if you want to perform a task only when the animation is ended, as shown in the following example:

```
private void OnStartAnimation(object sender,
    RoutedEventArgs e)
{
    EllipseAnimation.Begin();
    EllipseAnimation.Completed += (obj, args) =>
    {
```

```
        //the animation is ended  
    };  
}
```

WinUI greatly enhanced the basic XAML animation experience by introducing new features such as transitions, composited animations, and built-in control animations. We'll learn more about them in [\*Chapter 5, Designing Your Application\*](#). However, for the moment, it is important to introduce basic XAML animations, since they will play a significant role in the next section, where we'll discuss template customizations.

## Customizing the template of a control

Another feature that makes XAML so powerful is the ability to customize the template of every control. Thanks to this feature, you can completely change the look, feel, or behavior of a control without creating a new one, as you would need to do in Windows Forms.

The template of a control is stored in a property called **Template**, which you can redefine to change it. Let's see, for example, how we can customize the template of a **Button** control:

```
<Button Content="Ok" Background="Red" Width="150"  
    Height="50">  
    <Button.Template>  
        <ControlTemplate TargetType="Button">  
            <Grid>  
                <Ellipse Height="{TemplateBinding  
Height}"  
                    Width="{TemplateBinding  
Width}"  
                    Fill="{TemplateBinding  
Background}" />
```

```
<ContentPresenter
    HorizontalAlignment="{TemplateBinding
        HorizontalContentAlignment}"
    VerticalAlignment="{TemplateBinding
        VerticalContentAlignment}" />
</Grid>
</ControlTemplate>
</Button.Template>
</Button>
```

The key of the template is the **ContentPresenter** object, which represents a placeholder where the content will be placed. You can see, for example, that we have set the **Content** property of **Button** to a label, **Ok**. When the control is rendered, the label will be displayed where we have placed **ContentPresenter** in the template.

Around **ContentPresenter**, we can define the layout we prefer to redefine the look and feel of the control. In this example, we have added **ContentPresenter** together with an **Ellipse** control inside a **Grid** container. This enables us to create a rounded button, as shown in the following screenshot:

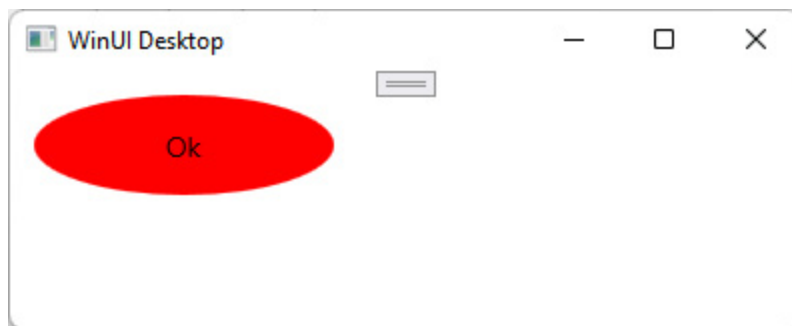


Figure 2.2 – A Button control with a custom template

Note the special **TemplateBinding** markup expression, which enables you to inherit the value from a property defined in the control itself – for example, you can see how the **Ellipse** control has a red background, since its **Fill** property is connected to the **Background** property of the **Button** control through the **TemplateBinding** expression.

If you want to customize the template of a control that is used to display a collection of data, such as **ListView**, there's a small difference – instead of **ContentPresenter**, you must use **ItemsPresenter**, which acts as a placeholder for the list of items assigned as the source. The following code shows a similar example to the previous one (using **Ellipse** as the background) but, this time, applied to a **ListView** control:

```
<ListView x:Name="lstPeople" Background="Red"
Width="300">
    <ListView.Template>
        <ControlTemplate>
            <Grid>
                <Ellipse Height="{TemplateBinding
Height}"
                        Width="{TemplateBinding
Width}"
                        Fill="{TemplateBinding
Background}" />
            <ScrollView>
                <ItemsPresenter />
            </ScrollView>
        </Grid>
    </ControlTemplate>
</ListView.Template>
</ListView>
```

In the previous example, the **Template** property has been customized directly on the control itself. However, as with any other property, you can also customize it through a style and declare it as an application's resource.

If you try the previous example code in your application, you might notice how, after customizing the template, the button doesn't behave as expected anymore – for example, the hover effect when you mouse over a button is gone.

The reason is that, by redefining the **Template** property, we have completely stripped it of all the extra properties that handle the various states the control can assume (**Pressed**, **Hover**, and so on). For this reason, in a real application, when you want to change the template of a control, you typically customize the default one rather than create a completely new one from scratch.

Unfortunately, this is an area where, at the time of writing, you will face some limitations compared to other XAML-based frameworks such as WPF or Universal Windows Platform. The reason is that, for these technologies, Visual Studio provides a designer, which gives you the possibility to design the UI of your application without launching it. Additionally, the designer offers an easy way to generate a copy of a control's template.

WinUI 3, at the moment, doesn't offer a designer, so you won't be able to use it to get access to the control's default template. Let's explore a couple of workarounds.

## Using a Universal Windows Platform project

The **Universal Windows Platform (UWP)** also uses WinUI to render the UI of an application. Unlike WinUI 3, Visual Studio supports the designer for UWP apps, so we can use an empty UWP project to copy the templates we need and import them into our Windows App SDK project.

The first step is to create a new UWP project, using the **Blank App (Universal Windows)** template that you'll find in Visual Studio. Once you have created it, double-click on the default page (the one called **MainPage.xaml**) and add the control you would like to customize in your original application – for example, let's add a **Button** control inside the page:



```
<Page>
  <Grid>
    <Button Content="Ok" />
  </Grid>
</Page>
```

At the bottom of the window, you will see a tab called **Design**, near another one called **XAML**. These two tabs can be used to switch between the **XAML** view (where you can see the code) and the **Design** view (where you can see the designer). Click on **Design** to see a preview of your application:

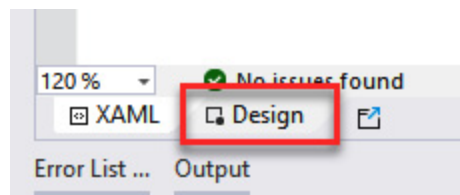


Figure 2.3 – The tab to access the designer in Visual Studio

Now, open the window called **Document Outline**. If you don't see it, you can find it by following the menu and going to **View | Other windows | Document Outline**. This window will show you the XAML tree of the current XAML page, which, in our case, will contain just the **Button** control we have added. Right-click on the button and choose **Edit Template | Edit a Copy...**:

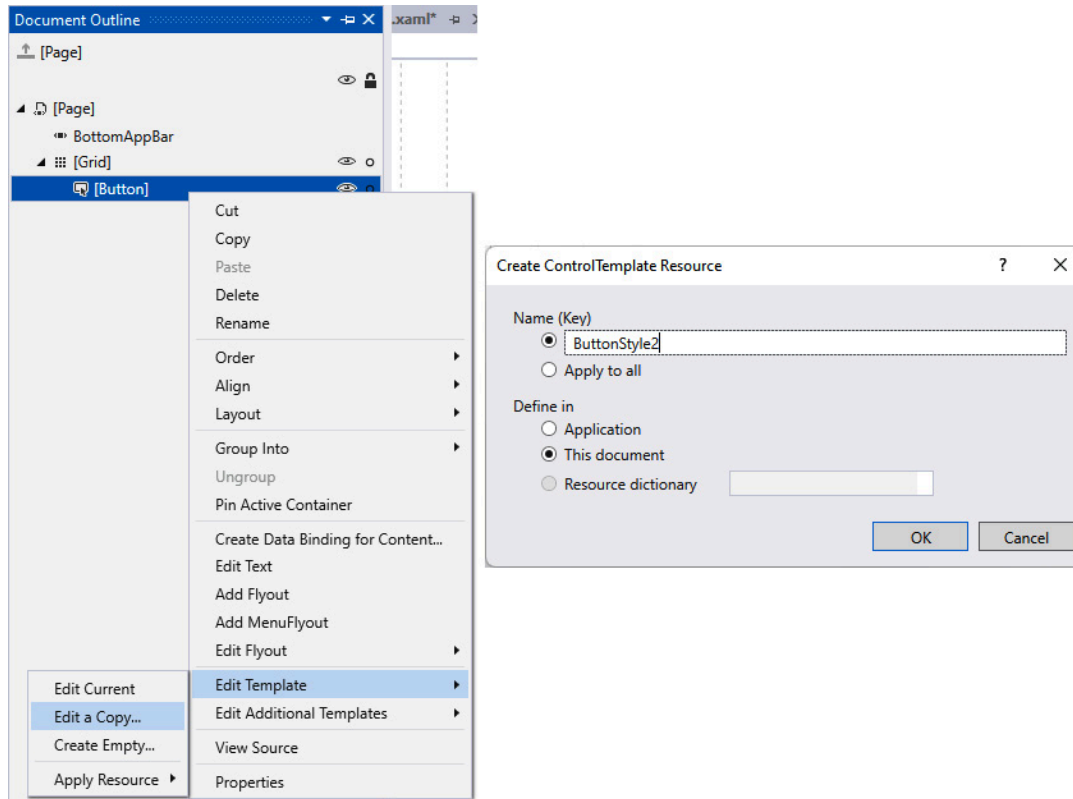


Figure 2.4 – The option in the Document Outline window to create a copy of a control's template

Give it a meaningful name (for example, **MyCustomButtonStyle**) and press **OK**. Now, go back to the **XAML** view. You will find, declared as a resource of the page, a new style with the whole template of the **Button** control, which, as you can see, is much more complex than the one we created before. Now, we just have to copy and paste this new style into our original Windows App SDK application and start customizing it.

## Using the XAML source code

The Windows 10 SDK, which is installed together with Visual Studio, includes a file called **generic.xaml**, which contains all the default styles and templates for the built-in controls. You can find this file inside the **C:\Program Files (x86)\Windows**

**Kits\10\DesignTime\CommonConfiguration\Neutral\UAP\10.0.22000.0\Generic** folder.

All the default templates are prefixed by the following comment:

```
<!--Default style for Windows.UI.Xaml.Controls.<name of
the
control>
```

Let's say, for example, that you want to customize the template of a **Button** control. In this case, you open the **generic.xaml** file with your favorite text editor and search for the **Windows.UI.Xaml.Controls.Button** string. Once you have found the full template of the control, you can copy and paste it into your Windows App SDK project to start customizing it:

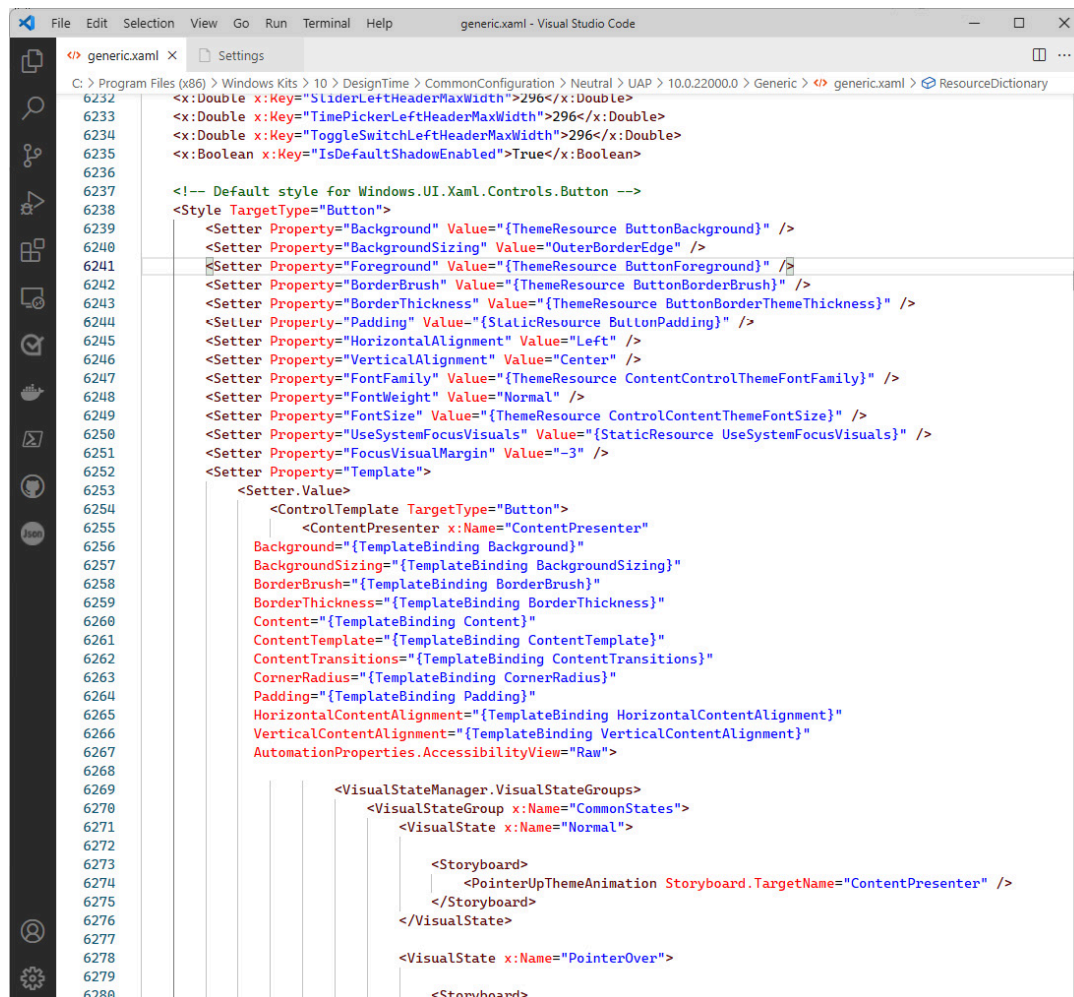


Figure 2.5 – The generic.xaml file opened in Visual Studio Code with the template of the Button control

Instead, if you want to customize the template of one of the most recent controls that were added since WinUI 2.x, you will find the tem-

plate in the GitHub repository of the project

(<https://github.com/microsoft/microsoft-ui-xaml/>). For example, let's say that you want to customize the template of the **InfoBar** control. In this case, you can look for the **InfoBar** folder at

<https://github.com/microsoft/microsoft-ui-xaml/tree/main/dev> and open the file called **InfoBar.xaml**. Then, you can copy the template definition into your Windows App SDK project and customize it:

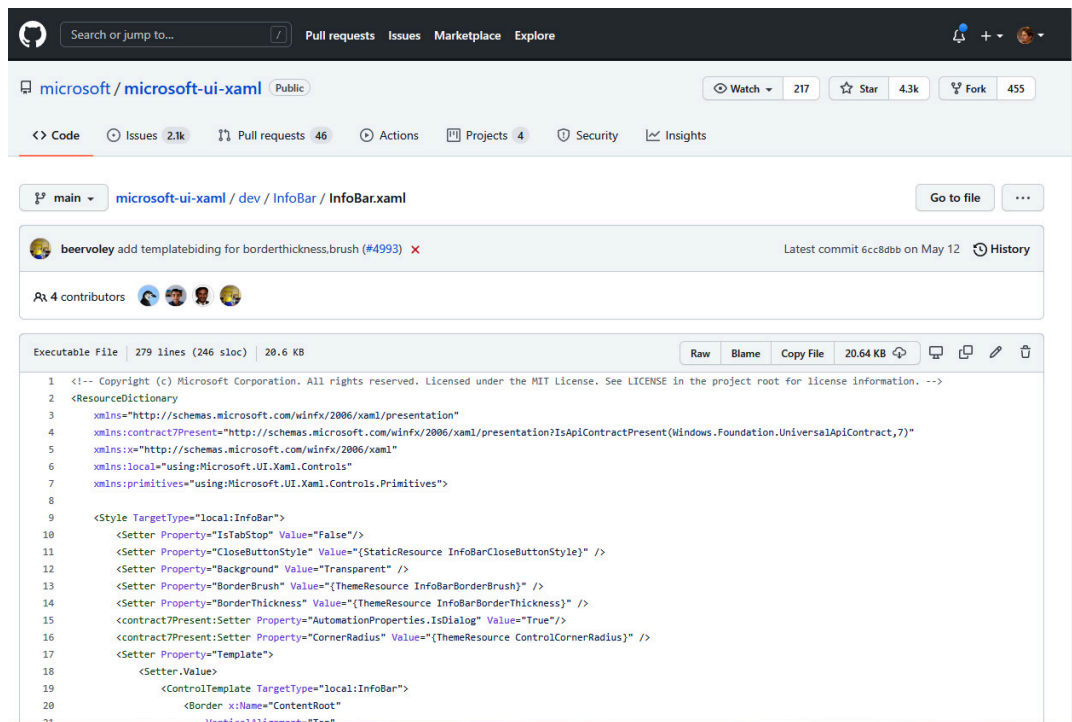


Figure 2.6 – A screenshot of the file on GitHub that contains the template of the InfoBar control

Regardless of your choice, if you take a deeper look at the template, you will notice that, compared to the original one we have created on our own, there's a new object we haven't met before, called **VisualStateManager**. This is the XAML feature that enables all the features that were missing in our custom template, such as the hover animation. Let's see a bit more about how it works.

## Managing the states of a control

In the previous section, we saw a glimpse of a new important concept in XAML – visual states. A visual state is a way to represent the look and feel of a control in a specific situation. For example, a **Button** control can be pressed, disabled, or highlighted. In all these states, **Button** has a different aspect than the default one, as you can see in the following screenshot:

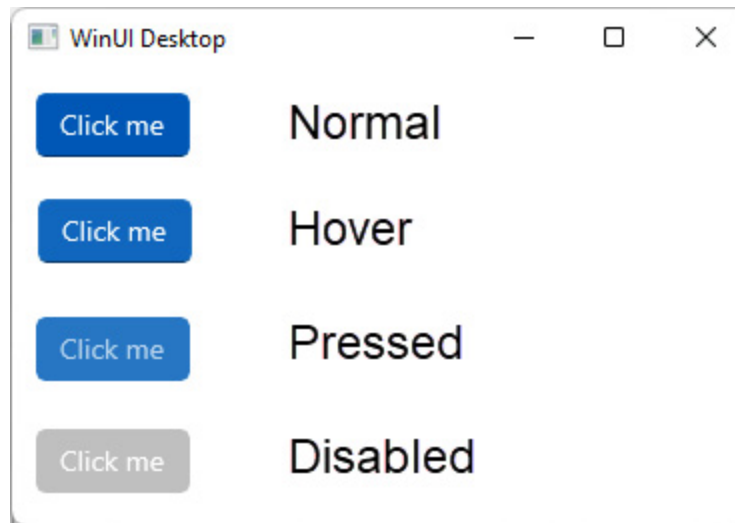


Figure 2.7 – The various states of a Button control

The power of visual states is that they simplify the management of the various states that a control can assume, since you don't have to specify the whole template each time. A visual state just specifies the differences compared to the default template of the control.

As an example, let's take a look at the **VisualState** definition called **Pressed** of a **Button** control:

```
<VisualState x:Name="Pressed">
  <Storyboard>
    <ObjectAnimationUsingKeyFrames
      Storyboard.TargetProperty="Fill"
      Storyboard.TargetName="BorderCircle">
      <DiscreteObjectKeyFrame KeyTime="0"
        Value="{ThemeResource
SystemControlBackground
```

```
        BaseMediumLowBrush}"/>
    </ObjectAnimationUsingKeyFrames>
    <ObjectAnimationUsingKeyFrames
        Storyboard.TargetProperty="Foreground"
        Storyboard.TargetName="ContentPresenter">
        <DiscreteObjectKeyFrame KeyTime="0"
            Value="{ThemeResource SystemControl
                HighlightBaseHighBrush}"/>
    </ObjectAnimationUsingKeyFrames>
    <PointerDownThemeAnimation
        Storyboard.TargetName="RootGrid"/>
</Storyboard>
</VisualState>
```

Instead of redefining the whole template of **Button**, the visual state is just changing a property on two controls included in the template – **BorderCircle** and **ContentPresenter**. By assigning a new color to these two properties, the visual state simulates the pressed effect.

Looking at the XAML code, it should be clear why we have previously introduced animations in this chapter. Each **VisualState** is defined with **Storyboard**, which includes one or more animations that change the value of the properties. As you can see, in most of the cases, these animations are created with the **UsingKeyFrame** variant, with **KeyTime** set to **0**. This makes the animation instantaneous, leading to an immediate state change.

The transition from one state to another is controlled by a class called **VisualStateManager**, which is typically used in combination with the events exposed by the control to trigger the switch from one state to another.

For example, consider the following XAML snippet:

```
<StackPanel>
    <VisualStateManager.VisualStateGroups>
```

```

        <VisualStateGroup x:Name="ButtonStates">
            <VisualState x:Name="NormalState" />
            <VisualState x:Name="HiddenState">
                <Storyboard>
                    <DoubleAnimation
                        Storyboard.TargetName="MyButton
"
                        Storyboard.TargetProperty="Opac
ity"
                        From="1" To="0"
                        Duration="0:0:3">
                    </DoubleAnimation>
                </Storyboard>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
    <Button Content="Start animation" x:Name="MyButton"
        Click="OnStartAnimation"
        PointerEntered="MyButton_PointerEntered"
        PointerExited="MyButton_PointerExited" />
</StackPanel>

```

Inside **VisualStateManager**, we have defined **VisualStateGroup** with two visual states – **NormalState** and **HiddenState**. You can see how **NormalState** doesn't include any storyboard. This means that **NormalState** matches the default state of the control. **HiddenState**, instead, contains **Storyboard** with an animation that hides the **Button** control included in the page, by changing its opacity from **1** to **0** in 3 seconds.

The **Button** control itself is subscribed to two events – **PointerEntered** (which is triggered when the mouse moves over the control) and **PointerExited** (which is triggered when the mouse moves outside the control). Let's take a look at the two event handlers in the code-behind:



```
private void MyButton_PointerEntered(object sender,
    PointerRoutedEventArgs e)
{
    VisualStateManager.GoToState(this, "HiddenState",
        false);
}
private void MyButton_PointerExited(object sender,
    PointerRoutedEventArgs e)
{
    VisualStateManager.GoToState(this, "NormalState",
        false);
}
```

The transition to another states is triggered by the **GoToState()** method of the **VisualStateManager** class, which belongs to the **Microsoft.UI.Xaml** namespace. As parameters, you must pass the following:

- A reference to the control that is going to change its visual state
- The name of the state
- A **boolean** value to tell if you want to apply the transition with an animation

If you try this code, you will see the button disappearing when you move the mouse over it (since you will move to the **HiddenState** visual state) and reappearing when you move the mouse outside it (since you will move to the **NormalState** visual state, which is the default look and feel).

The code you've just seen is an example to demonstrate the usage of **VisualStateManager**, but it isn't very different from the actual approach used by every control in the framework to manage their different states. For example, if you look at the implementation details of the **Button** control, you will see that it uses **VisualStateManager** to move to the **Pressed** state when the **Click** event is raised or to the **PointerOver** state when the **PointerEntered** event is raised.



WinUI has introduced a new powerful way to handle visual states – triggers. Thanks to triggers, you don't have to manually subscribe to events and use the **VisualStateManager.GoToState()** API, and instead can focus only on defining the differences between one visual state and the other. We'll cover this topic in ***Chapter 5**, Designing Your Application*, since triggers play a key role in building responsive UIs.

## Data binding

Data binding is one of the most important concepts in XAML, and it's the key to mastering many of the topics that we're going to discuss in the next chapters.

Thanks to data binding, you can create a communication channel between two different properties (either in code or in XAML) so that they get notified every time there's an update. This feature enables us to manage scenarios such as multiple controls that are connected to each other, or to create UIs that can automatically react to changes in a data layer. Data binding also enables us to define a clear separation between the UI layer and the data layer, and it's actually at the heart of the **Model-View-ViewModel (MVVM)** pattern, which we'll cover in ***Chapter 6**, Building a Future-Proof Architecture*.

Binding involves two actors – the source (which is the data layer) and the target (which is the UI control that will display the information). Let's consider the following example:

```
<StackPanel>
    <Slider x:Name="MySlider" />
    <TextBox Text="{Binding ElementName=MySlider,
        Path=Value}" />
</StackPanel>
```

We're using data binding to connect two controls – **Slider** and **TextBox**. The binding channel is created using the **Binding** markup ex-

pression. In this context, the binding is made by the following components:

- The **Slider** control is the source object. It's the one that generates the data that we want to display. We set it using the **ElementName** property of the binding expression.
- The **TextBox** control is the target object. It's the one that will display the value coming from the **Slider** control.
- The **Value** property of the **Slider** control is the source path. **Slider**, as with all the other XAML controls, has multiple properties, so we must specify which one we want to connect to the binding channel. We set it using the **Path** property of the binding expression.
- The **Text** property of the **TextBox** control is the target dependency property. This is the property that we want to set with the value coming from the source.

If you run this code, you will observe that the two controls are now connected – when we move the slider, **TextBox** will display the slider's value in real time. This is possible thanks to the fact that every property exposed by an XAML control is a dependency property. These are special properties that, other than being able to store a value, can also send a notification to the other side of the binding channel every time the value changes. Thanks to this notification, the UI control knows that it must update its state to reflect the new value.

Binding can support different modes. The default one is **OneWay**, which is automatically applied when you don't specify the **Mode** property of a binding expression (as in the previous example). In this mode, the source can update the target but not the other way around. If you test the previous code, you will see that moving the slider will change the value displayed by **TextBox**; however, if you manually type a new value in **TextBox**, the **Slider** control won't move. Typically, you're going to use **OneWay** communication when you're using, as a target, a control that is only capable of data output, such as **TextBlock**.

If you want to enable two-way communication, you must explicitly set **Mode** to **TwoWay**, as shown in the following example:

```
<StackPanel>
    <Slider x:Name="MySlider" />
    <TextBox Text="{Binding ElementName=MySlider,
        Path=Value, Mode=TwoWay}" />
</StackPanel>
```

You can now see that typing a new value in **TextBox** will change the position of the **Slider**. You typically use **TwoWay** binding with all the controls that enable you to input some data, such as **TextBox** or **DatePicker**.

In the end, there's also **OneTime** mode, which is helpful when you're using binding to set up the initial status of your controls but you aren't planning to have real-time updates. Binding can be expensive from a performance point of view, so it's a good choice to disable real-time notifications if you know in advance that the binding expression will never change. As we're going to learn in ***Chapter 6**, Building a Future-Proof Architecture*, this mode is quite helpful when you use the MVVM pattern, since you're going to use binding to connect any kind of data to the UI.

Binding expressions are evaluated at runtime, which might lead to performance issues, challenges in finding errors, and so on. To mitigate these problems, WinUI has introduced a new markup expression called **x:Bind**, which we'll explore in the next chapter.

## Binding with C# objects

In the previous section, we learned about element binding, in which the source is the property of another XAML control. However, most of the time, the data you need to display in the UI is defined in code. It can be a collection retrieved from a REST API or the result of a query executed on a SQL database. In all of these scenarios, you typically

have C# objects that you want to display, using one or more XAML controls.

To support this scenario with data binding, we have to introduce another important concept – **DataContext**. It's a special property, exposed by every XAML control, which defines the binding context. The XAML framework will try to resolve a binding expression by looking for the source and the source path in the binding context you have specified. In the previous example, we didn't need to explicitly set it. Thanks to the **ElementName** attribute, XAML knew where to find the **Value** property that we defined as the source path. In case you need to use C# objects instead as a data source, we must tell the runtime where it can find them.

Let's start with an example, by introducing the **Person** class:

```
public class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public int Age { get; set; }
}
```

Now, let's connect these properties to our XAML page using binding:

```
<StackPanel x:Name="MyPanel" >
    <TextBlock Text="{Binding Path=Name}" />
    <TextBlock Text="{Binding Path=Surname}" />
    <TextBlock Text="{Binding Path=Age}" />
</StackPanel>
```

Out of the box, however, WinUI doesn't know where to find these properties. As such, we have to manually set **DataContext** in the code-behind, as shown in the following example:

```
private void Page_Loaded(object sender, RoutedEventArgs
e)
```

```
{  
    Person = new Person  
    {  
        Name = "Matteo",  
        Surname = "Pagani",  
        Age = 38  
    };  
    MyPanel.DataContext = person;  
}
```

We create a new instance of the **Person** class and set it as **DataContext** of the **StackPanel** control. One of the most powerful features of the **DataContext** property is that it's hierarchical. By setting it on the **StackPanel** control, every child control will be able to access its properties. For this reason, it's common practice to set an object as **DataContext** of the entire page so that every control in it will be able to use its properties.

This example, however, has a flaw – if you try to update one of the properties of the **Person** object after setting it as **DataContext**, you won't see the new values showing up in the UI. Let's learn how to solve this problem.

## Implementing the **INotifyPropertyChanged** interface

When we started talking about data binding in this chapter, we saw an example in which we connected two XAML controls using a binding channel. In this scenario, updates were displayed in real time, since every property exposed by XAML controls is a dependency property that can send notifications to the channel every time the value changes.

In the previous example, the **Person** class is exposing normal properties instead. Therefore, if you change one of its values after setting it as

**DataContext**, you won't see them in the UI – the property isn't notifying the binding channel that its value has changed. However, dependency properties can be quite complex to declare and maintain, and as such, they aren't a good fit for plain C# objects.

To support this scenario, XAML offers an interface called **INotifyPropertyChanged** that your C# classes can implement to support sending notifications when one of their properties changes values. The following example shows how our **Person** class can be refactored to support this interface:

```
public class Person: INotifyPropertyChanged
{
    private string _name;
    public string Name
    {
        get { return _name; }
        set
        {
            _name = value;
            OnPropertyChanged();
        }
    }
    public string Surname { get; set; }
    public int Age { get; set; }
    public event PropertyChangedEventHandler
        PropertyChanged;
    private void OnPropertyChanged([CallerMemberName]
        string propertyName = "")
    {
        PropertyChanged?.Invoke(this, new
            PropertyChangedEventArgs(propertyName));
    }
}
```

By implementing the **INotifyPropertyChanged** interface, we must expose an event called **PropertyChanged**, which we must invoke every time one of the properties of the class changes. Therefore, as part of the refactoring, we have also to change the way we define our properties. Instead of using automatic properties, we need to explicitly define the **get** and **set** so that when the value of the property changes, we invoke the **OnPropertyChanged()** method.

Now, our class is fully ready to act as a source for data binding. If we change the **Name** property of the **Person** class, the XAML control that acts as a target for the binding channel will update its status to display the new value.

In ***Chapter 6**, **Building a Future-Proof Architecture***, we'll learn how, in a real application, it's unlikely that you will have to implement the **INotifyPropertyChanged** interface on your own (as shown in the previous example). Most of the toolkits and frameworks that help you implement the MVVM pattern provide a base class that does all the heavy lifting for you.

With this understanding, we'll move on to the next section to see how to display data collections.

## Displaying collections of data with DataTemplates

Another powerful type of resource is **DataTemplates**, which you can use to customize the look and feel of a control. As the name says, these templates are supported by controls that you can use to display collections of data, such as **ListView** or **GridView**. Thanks to **DataTemplate**, you can define the look and feel of a single item of the collection, which will then be applied to all the items.

This is an example of what **DataTemplate** looks like:

```
<ListView>
  <ListView.ItemTemplate>
    <DataTemplate>
      <StackPanel>
        <TextBlock Text="{Binding Path=Name}"
      />
        <TextBlock Text="{Binding
Path=Surname}" />
      </StackPanel>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

By looking at the code, you'll probably understand why we are talking about this type of resource only now. As you can see from the snippet, **DataTemplate** uses the **Binding** markup expression to connect the various controls inside the template with the data we want to display.

Under the hood, the **DataContext** property of the template becomes a single item from the collection. In this case, we're assuming that we want to use this template to display a collection of people, which are mapped by the following class:

```
public class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
}
```

The output of the example is that, for each **Person** object included in the collection displayed by the **ListView** control, XAML will render a panel with the name and the surname of the person.

In the previous snippet, **DataTemplate** was defined inline, but it can be treated like a regular resource by assigning the **x:Key** property, as shown in the following example:

```
<StackPanel>
```



```
<StackPanel.Resources>
    <DataTemplate x:Key="PeopleTemplate">
        <StackPanel>
            <TextBlock Text="{Binding Path=Name}"
        />
            <TextBlock Text="{Binding
Path=Surname}" />
        </StackPanel>
    </DataTemplate>
</StackPanel.Resources>
<ListView ItemTemplate="{StaticResource
    PeopleTemplate}" />
</StackPanel>
```

With the standard **DataTemplate** configuration, the same template will be applied to all the items in the collection. What if, instead, you want to apply a different template based on a condition? Let's say that, in the previous example, we wanted to use a different template based on the age of the person to highlight people who are below 18 years old. We can support this scenario thanks to a special XAML feature called **DataTemplateSelector**.

The first step to implement this scenario is to create a new class that contains the logic we need to differentiate the various templates. The following example shows the class implementation for the age scenario:

```
public class PeopleTemplateSelector:
    DataTemplateSelector
{
    public DataTemplate YoungTemplate { get; set; }
    public DataTemplate AdultTemplate { get; set; }
    protected override DataTemplate
        SelectTemplateCore(object item)
    {
        Person = item as Person;
```

```
        if (person.Age < 18)
        {
            return YoungTemplate;
        }
        else
        {
            return AdultTemplate;
        }
    }
}
```

The first step is to inherit our class from the **DataTemplateSelector** class, which is included in the **Microsoft.UI.Xaml.Controls** namespace.

Then, we need to define a property for each template we need to support (which are defined using the **DataTemplate** class). In our scenario, we need only two of them – one for young people and one for adults.

In the end, we must override the **SelectTemplateCore** method, which gets invoked when the control that is going to use this selector needs to render an item. In this method, we can add logic that will return one **DataTemplate** object or another, based on the condition we have decided. As a parameter, we get a reference to the item of the collection that is being evaluated. Before using it in a meaningful way, however, we need to cast it first to the data type that we're actually displaying (in our scenario, it's a **Person** object). In this example, we'll return the template called **YoungTemplate** if the person's age is below 18; otherwise, we return the one called **AdultTemplate**.

Now that we have defined **DataTemplateSelector**, we can go back to XAML and define, among our resources, the two templates we need – **AdultTemplate** and **YoungTemplate**:

```
<StackPanel>
    <StackPanel.Resources>
```

```

        <DataTemplate x:Key="YoungTemplate">
            <Border BorderBrush="Red"
BorderThickness="3">
                <StackPanel>
                    <TextBlock Text="{Binding
Path=Name}"
                        />
                    <TextBlock Text="{Binding
                        Path=Surname}" />
                </StackPanel>
            </Border>
        </DataTemplate>
        <DataTemplate x:Key="AdultTemplate">
            <Border BorderBrush="Green"
BorderThickness="3">
                <StackPanel>
                    <TextBlock Text="{Binding
Path=Name}"
                        />
                    <TextBlock Text="{Binding
                        Path=Surname}" />
                </StackPanel>
            </Border>
        </DataTemplate>
    </StackPanel.Resources>
</StackPanel>

```

As a final step, we need to declare the custom class that we have created as a resource as well, as shown in the following example:

```

<StackPanel>
    <StackPanel.Resources>
        <local:PeopleTemplateSelector
            x:Key="PeopleTemplateSelector"
            AdultTemplate="{StaticResource
                AdultTemplate}"

```

```

        YoungTemplate="{StaticResource
        YoungTemplate}" />
    <DataTemplate x:Key="YoungTemplate">
        <Border BorderBrush="Red"
BorderThickness="3">
            <StackPanel>
                <TextBlock Text="{Binding
Path=Name}"
                    />
                <TextBlock Text="{Binding
                    Path=Surname}" />
            </StackPanel>
        </Border>
    </DataTemplate>
    <DataTemplate x:Key="AdultTemplate">
        <Border BorderBrush="Green"
BorderThickness="3">
            <StackPanel>
                <TextBlock Text="{Binding
Path=Name}"
                    />
                <TextBlock Text="{Binding
                    Path=Surname}" />
            </StackPanel>
        </Border>
    </DataTemplate>
</StackPanel.Resources>
<ListView ItemTemplateSelector="{StaticResource
    PeopleTemplateSelector}"
    x:Name="lstPeopleWithSelector"
    />
</StackPanel>

```

Note how, in this case, we had to add the namespace prefix before declaring the **PeopleTemplateSelector** object; since it's a custom class we have created, it isn't part of the standard WinUI framework. As

part of the definition, we connect the two **AdultTemplate** and **YoungTemplate** properties with the corresponding **DataTemplate** objects that we previously declared.

In the end, we assign the **PeopleTemplateSelector** resource to the **ItemTemplateSelector** property exposed by the **ListView** control. Note how, in this scenario, we no longer need to assign a template to the **ItemTemplate** property.

Let's populate this list now with a few values in the code-behind:

```
private void Page_Loaded(object sender, RoutedEventArgs e)
{
    List<Person> people = new List<Person>
    {
        new Person { Name = "Matteo", Surname =
        "Pagani",
            Age = 38},
        new Person { Name = "Marc", Surname =
        "Plogas",
            Age = 27},
        new Person { Name = "John", Surname = "Doe",
            Age = 15 }
    };
    PeopleList.ItemsSource = people;
}
```

This is what the final result looks like:

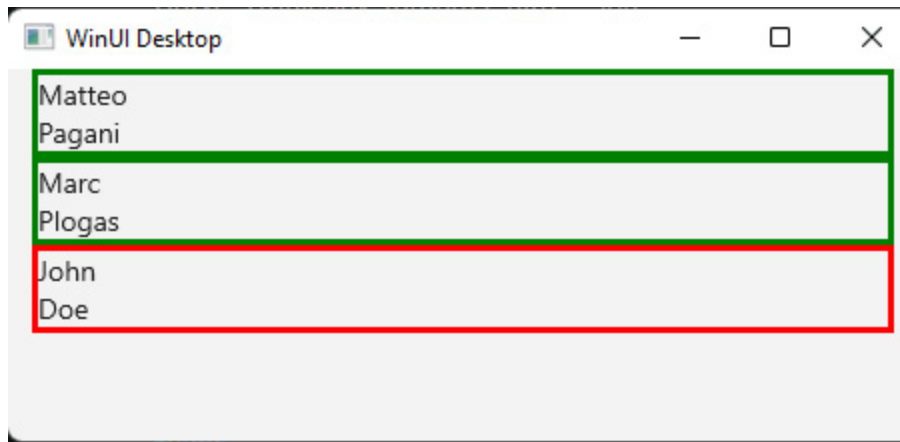


Figure 2.8 – A ListView control that uses an ItemTemplateSelector component

The first two items in the list are displayed using the **AdultTemplate** resource, since the **Age** property for both is higher than 18. The last item is rendered instead using the **YoungTemplate** resource, since the **Age** property is lower than 18.

In the next section, we will be seeing how to work with user controls.

## Creating user controls

WinUI, like Windows Forms, supports the option to create user controls, which is a way to reuse UI components across an entire application. A user control is structured like a page – it has an XAML file, which contains existing XAML controls, and a code-behind file, which includes the logic. For example, let's say that your application offers the option to start a search. You can create a user control specific for this task, which groups together the UI (**TextBox** to write the search term, **Button** to start the search, and so on) and the logic required to perform a search across your data.

An important difference between WinUI and Windows Forms is the way you expose properties. Typically, a user control exposes one or more properties to allow customization when you add it to a specific page of your application. For example, the search box might have a

different title based on the page where it's placed. As such, the title can't be hardcoded, but it must be exposed as a property so that we can customize it without being forced to create multiple copies of the same control.

In Windows Forms, you can expose them as normal properties, but in WinUI, you must expose them as dependency properties; otherwise, you won't be able to properly support binding, making the user control less flexible.

Let's start from the beginning and assume we want to build a custom control to render a search box. This control is made by **Label1**, which acts as a header; **TextBox**, where the user can type the search keyword; and a **Button** control, which starts the search.

The first step is to right-click on your WinUI project in Visual Studio and choose **Add | New item**. Look for the template called **User Control (WinUI 3)**, give it a meaningful name (for example, **SearchBox**), and press **Add**.

As already mentioned, the user control is structured like a regular WinUI page. The only difference is that the base class isn't **Page** or **Window** but **UserControl1**. Therefore, you can add all the elements you need to define the look and feel of your built-in and third-party XAML controls, styles, animations, and so on. For example, this is how we can create a user control in XAML:

```
<UserControl>
  <StackPanel>
    <TextBlock x:Name="txtSearchHeader" />
    <TextBox PlaceholderText="Search..."
      x:Name="txtSearchText" />
    <Button Content="Search" Click="OnSearch" />
  </StackPanel>
</UserControl>
```

The code-behind file, instead, will look like this:

```
public sealed partial class SearchBox : UserControl
{
    public SearchBox()
    {
        this.InitializeComponent();
    }
    private void OnSearch(object sender,
RoutedEventArgs e)
    {
        //start the search
    }
}
```

As we would do on a regular page, we're going to use the **OnSearch()** event handler to add the logic required to start a search.

So far, there aren't many differences compared to a Windows Forms user control, except for the usage of XAML to define the UI.

Let's move on and focus on **TextBlock**, which acts as a header of the box. In this case, we aren't setting a fixed text, since we want to customize it based on the page where this user control will be placed. As such, we'll need to expose a new property called **HeaderText**, which will internally update the **Text** property of our **TextBlock** control. As mentioned, we can't expose it as a normal property, like in the Windows Forms user control; otherwise, we won't be able to use it with data binding. We have to expose it as a dependency property, as shown in the following example:

```
public string HeaderText
{
    get { return (string)GetValue(HeaderTextProperty); }
    set { SetValue(HeaderTextProperty, value); }
```



```
}  
public static readonly DependencyProperty  
    HeaderTextProperty =  
        DependencyProperty.Register("HeaderText",  
            typeof(string), typeof(SearchBox), new  
                PropertyMetadata(string.Empty));
```

The **HeaderText** property is backed by a **HeaderTextProperty** object, which is the **DependencyProperty** type. These special properties are registered by calling the **DependencyProperty.Register()** method and passing the following parameters:

- The name of the property (in our case, **HeaderText**).
- The property's type (in our case, it's **string**).
- The control that exposes this property (in our case, it's our **SearchBox** user control). Typically, it's the class itself that is registering the property.
- A **PropertyMetadata** object, which holds some extra information about the property. In this first example, the only information we pass is the default value if the property isn't set, which is an empty string in our scenario.

Now, our control exposes a **HeaderText** property that fully supports binding, styles, and so on. However, we still miss a step. We have defined the property, but we aren't managing the fact that, when this property is set, our **TextBlock** control should display the value of the **HeaderText** property. We can achieve this goal by adding extra information to the **PropertyMetadata** object, which is a callback method that we want to invoke when its value changes. Here is the full implementation:

```
public string HeaderText  
{  
    get { return (string)GetValue(HeaderTextProperty);  
    }  
    set { SetValue(HeaderTextProperty, value); }  
}
```

```
}  
public static readonly DependencyProperty  
    HeaderTextProperty =  
        DependencyProperty.Register("HeaderText",  
            typeof(string), typeof(SearchBox), new  
                PropertyMetadata(string.Empty,  
                    OnHeaderTextChanged));  
private static void  
OnHeaderTextChanged(DependencyObject d,  
    DependencyPropertyChangedEventArgs e)  
{  
    var searchBox = d as SearchBox;  
    var header = searchBox.FindName("txtSearchHeader")  
as  
    TextBlock;  
    header.Text = e.NewValue.ToString();  
}
```

We have added to the **PropertyMetadata** object a reference to the **OnHeaderTextChanged()** event handler, which will be invoked every time the value of the property changes. Inside this handler, we're going to change the **Text** property of our **TextBlock** with the new value, which we can retrieve from the **NewValue** property stored in the event arguments.

However, you can see that the code to change the **Text** property isn't as straightforward as we would expect to see. The reason is that **DependencyProperty** is a static object, and as such, we can't directly access the controls declared in the XAML, which aren't static.

The workaround is to use the **DependencyObject** parameter of the event handler, which holds a reference to the object that is exposing the property (our **SearchBox** control). After having converted it to a **SearchBox** object, we can use the **FindName()** method to retrieve a reference to our **TextBlock** control by passing as a parameter its name

(**txtSearchHeader**). Finally, after casting it to the control type we're expecting (**TextBlock**, in this case), we can access all of its properties, including the **Text** one we were searching for.

Now, we are ready to include our **SearchBox** control in a page and customize its header by setting the **HeaderText** property, as shown in the following example:

```
<local:SearchBox HeaderText="This is my header" />
```

Since **HeaderText** is a dependency property, we can also safely set it using a binding expression:

```
<local:SearchBox HeaderText="{Binding  
Path=MyHeaderText}" />
```

Remember that since the **SearchBox** control is a custom one, we need to prefix it with the proper namespace when we add it to a XAML page.

## Summary

It's been quite a ride! XAML is a very powerful language and, through this chapter, we have seen some of its important features, such as resources, styles, and binding. Thanks to the knowledge you have acquired so far, you will be able to start designing complex UI interfaces and build real applications using WinUI and the Windows App SDK. If you are a Windows Forms developer, this chapter should have helped you also to understand how many of its concepts are transferrable to XAML and WinUI.

However, what we have learned so far are only the basic concepts. On top of them, WinUI has added many new exciting features, such as transitions, composite animations, and responsive layouts. We'll explore many of them in ***Chapter 6, Building a Future-Proof Architecture***, in which we'll learn how to design great WinUI experiences. In

***Chapter 7***, *Migrating Your Windows Applications to the Windows App SDK and WinUI*, we'll put into practice many of the concepts we've learned so far to migrate a real Windows Forms application.

The next chapter is dedicated to developers who are building WPF, and we'll learn how to translate some of the existing WPF concepts to WinUI. However, even if you haven't worked with WPF before, you might still learn some important concepts, so I suggest you don't skip it!

## Questions

1. Which is the best approach to adopt when you want to enable data binding using your C# classes as a data source?
2. The best place to declare a style is close to the control that is going to use it – true or false?
3. By default, when you create a binding channel, both sides (source and target) are notified whenever one of the values changes – true or false?